

Bases de datos relacionales

Aquí empieza lo bueno: cómo se monta de verdad una base de datos, tabla a tabla, con sus reglas, sus candados y quién puede tocarla.

20 horas

RA2

Módulo 0484

El modelo relacional, explicado en fácil

El modelo relacional usa palabras muy formales para cosas que ya conoces. Aquí tienes la traducción.

- Relación = Tabla: sí, la típica cuadrícula con filas y columnas que ya has visto mil veces en Excel.
- Tupla = Fila / registro: cada línea de la tabla, por ejemplo, los datos de un alumno.
- Atributo = Columna / campo: cada dato concreto que guardas (nombre, DNI, nota...).
- Dominio: los valores que puede tomar ese campo (un DNI siempre sigue el mismo patrón, no puede ser cualquier cosa).
- Grado: cuántas columnas tiene la tabla. Cardinalidad: cuántas filas tiene ahora mismo.

Identifica cada pieza del modelo relacional

Imagina esta tabla: *ALUMNO(id_alumno, nombre, id_ciclo)*, con 30 filas cargadas ahora mismo. Responde antes de mirar la solución.

- 1 ¿Cómo se llama en el modelo relacional lo que coloquialmente llamamos "la tabla ALUMNO" entera?
- 2 ¿Cómo se llama cada fila individual, por ejemplo los datos del alumno con *id_alumno = 5*?
- 3 ¿Cómo se llama cada columna, por ejemplo el campo "nombre"?
- 4 ¿Cuál es el grado de esta tabla? ¿Y su cardinalidad ahora mismo?

SOLUCIÓN

1. Relación: es el nombre formal de "la tabla completa".
2. Tupla (o registro): cada fila es una tupla.
3. Atributo (o campo): cada columna es un atributo.
4. Grado = 3 (tiene 3 columnas: *id_alumno*, *nombre*, *id_ciclo*). Cardinalidad = 30 (tiene 30 filas cargadas).

Las reglas que debe cumplir una tabla bien diseñada

El modelo relacional no es solo "filas y columnas": para que una tabla sea una relación de verdad, tiene que cumplir unas normas concretas.

- No puede haber dos filas exactamente iguales: siempre tiene que haber algo (normalmente la clave primaria) que distinga una fila de otra.
- El orden de las filas no importa: no existe "la primera fila" desde el punto de vista lógico, aunque el SGBD las devuelva en algún orden concreto.
- El orden de las columnas tampoco importa: identificas un dato por el nombre de su columna, no por su posición.
- Cada celda debe guardar un único valor atómico: nada de meter "Matemáticas, Lengua, Inglés" en una sola casilla de texto — eso rompe el modelo y nos dará problemas en el capítulo 6 (normalización).
- Este último punto es el que más se salta la gente al empezar, así que apúntatelo: una celda, un valor.

Tipos de datos: la chuleta completa

Cada columna necesita un tipo de dato. Elegir bien desde el principio te ahorra líos después.

Tipo	Para qué lo usas	Ejemplo
INT	Números enteros: identificadores, edades, cantidades	id_alumno INT
DECIMAL(p,s)	Números con decimales exactos: notas, precios, importes	nota DECIMAL(4,2)
VARCHAR(n)	Texto corto de longitud variable, hasta n caracteres	nombre VARCHAR(50)
TEXT	Texto largo sin límite práctico: descripciones, comentarios	observaciones TEXT
DATE / DATETIME	Fechas, o fechas con hora incluida	fecha_nac DATE
BOOLEAN	Verdadero o falso: sí/no, activo/inactivo	activo BOOLEAN

La sentencia CREATE TABLE

- Con CREATE TABLE creas la tabla entera de una sola vez: nombre, columnas, tipos y restricciones.
- El orden de las columnas en la sentencia es el orden en que quedarán guardadas, aunque —como hemos visto— eso no debería importarte al consultarlas.
- FOREIGN KEY se declara al final, después de todas las columnas, y apunta a la clave primaria de otra tabla que debe existir previamente.

CREATE TABLE

```
CREATE TABLE alumno (  
    id_alumno    INT            PRIMARY KEY,  
    nombre       VARCHAR(50)    NOT NULL,  
    apellidos    VARCHAR(80)    NOT NULL,  
    fecha_nac    DATE,  
    id_ciclo     INT,  
    FOREIGN KEY (id_ciclo)  
        REFERENCES ciclo(id_ciclo)  
);
```

Diseña tu propia tabla

Un ciclo formativo necesita guardar sus MÓDULOS: cada módulo tiene un código (por ejemplo "0484"), un nombre, unas horas anuales y pertenece a un único ciclo.

EJERCICIO

Escribe el CREATE TABLE de la tabla modulo, con el tipo de dato correcto para cada campo y la clave ajena hacia ciclo. Piensa tu respuesta antes de mirar la solución.

SOLUCIÓN

```
CREATE TABLE modulo (  
  codigo      VARCHAR(4)    PRIMARY KEY,  
  nombre      VARCHAR(80)   NOT NULL,  
  horas       INT           NOT NULL,  
  id_ciclo    INT,  
  FOREIGN KEY (id_ciclo)  
    REFERENCES ciclo(id_ciclo)  
);
```

Claves primarias, ajenas e índices

Sin claves, tu base de datos sería un caos de filas duplicadas que no se sabría cómo conectar entre sí.

Elemento	Para qué sirve
Clave primaria (PRIMARY KEY)	El DNI de cada fila: la identifica sin ninguna duda y no se puede repetir ni dejar vacía
Clave ajena (FOREIGN KEY)	El "puente" que conecta una tabla con otra, apuntando a su clave primaria
Índice	Un atajo para encontrar filas rápido, como el índice de un libro
Valor NULL	"Aquí no hay nada", que no es lo mismo que un 0 o una casilla vacía — y ojo, porque cambia cómo se comparan y suman los datos

Claves primarias compuestas

- A veces ningún campo por sí solo identifica una fila, pero la combinación de dos sí — es la clave primaria compuesta.
- Caso típico: una tabla intermedia de matrículas, donde lo único que identifica una fila es "este alumno, en este módulo" juntos.
- Se declara metiendo los dos campos dentro del mismo PRIMARY KEY, no dos PRIMARY KEY por separado (eso no está permitido).

Clave primaria compuesta

```
CREATE TABLE matricula (  
    id_alumno    INT,  
    id_modulo    VARCHAR(4),  
    fecha        DATE NOT NULL,  
    PRIMARY KEY (id_alumno, id_modulo),  
    FOREIGN KEY (id_alumno)  
        REFERENCES alumno(id_alumno),  
    FOREIGN KEY (id_modulo)  
        REFERENCES modulo(codigo)  
);
```

Encuentra la clave correcta

Para cada tabla, decide si necesita una clave primaria simple o una compuesta, y cuál sería.

- 1 Una tabla PROFESOR con DNI, nombre y departamento.
- 2 Una tabla INSCRIPCION que relaciona alumnos con actividades extraescolares: un alumno puede apuntarse a varias actividades, y una actividad tiene varios alumnos.
- 3 Una tabla ASIENTO_CINE que guarda qué fila y qué columna ocupa cada butaca dentro de una sala concreta.
- 4 Una tabla LIBRO con ISBN, título y autor.

SOLUCIÓN

1. Simple: el DNI ya identifica sin ambigüedad a cada profesor.
2. Compuesta: (id_alumno, id_actividad) juntos identifican cada inscripción — ninguno por separado basta.
3. Compuesta: (id_sala, fila, columna) — la misma fila y columna se repite en cada sala, así que hace falta combinarlas con la sala.
4. Simple: el ISBN ya es único para cada libro.

Restricciones de validación

Las restricciones son como un portero de discoteca para tus datos: si no cumples la norma, no entras a la tabla.

- NOT NULL: obliga a rellenar esa columna sí o sí.
- UNIQUE: no deja repetir un valor, aunque no sea la clave primaria (por ejemplo, dos alumnos no pueden tener el mismo email).
- CHECK: exige que se cumpla una condición (por ejemplo, que la nota esté entre 0 y 10).
- DEFAULT: pone un valor automático si no le dices nada al insertar.
- Cuanto más control pongas aquí, menos tendrás que estar comprobando "a mano" en cada aplicación que use la base de datos.

Un ejemplo real con varias restricciones combinadas

Así se ve una tabla "blindada" de verdad, combinando NOT NULL, UNIQUE, CHECK y DEFAULT a la vez.

Tabla con restricciones combinadas

```
CREATE TABLE alumno (  
  id_alumno    INT          PRIMARY KEY,  
  nombre       VARCHAR(50)  NOT NULL,  
  email        VARCHAR(80)  UNIQUE NOT NULL,  
  nota_media   DECIMAL(4,2) CHECK (nota_media BETWEEN 0 AND 10),  
  activo       BOOLEAN      DEFAULT TRUE,  
  id_ciclo     INT          NOT NULL,  
  FOREIGN KEY (id_ciclo) REFERENCES ciclo(id_ciclo)  
);
```

Fíjate: email es NOT NULL y UNIQUE a la vez (obligatorio y sin duplicados), nota_media solo acepta valores entre 0 y 10, y activo se pone en TRUE automáticamente si no dices lo contrario.

¿Qué restricción le falta a esta tabla?

Mira esta tabla de reservas de una biblioteca y decide qué restricción añadirías a cada campo, y por qué.

- 1 dni_socio VARCHAR(9) — dos socios distintos se han registrado con el mismo DNI por error. ¿Qué restricción lo habría evitado?
- 2 fecha_devolucion DATE — se ha guardado una reserva sin fecha de devolución y ha dado problemas en la aplicación. ¿Qué le falta?
- 3 estado VARCHAR(10) — alguien ha guardado el valor "pendiente" con la P mayúscula, luego "Pendiente", y ahora hay tres formas distintas de escribir lo mismo. ¿Qué restricción ayudaría (aunque no lo arregle del todo)?
- 4 dias_prestamo INT — se ha colado un préstamo con -5 días, algo que no tiene sentido. ¿Qué le falta?

SOLUCIÓN

1. UNIQUE: impediría que dos filas tuvieran el mismo DNI.
2. NOT NULL: obligaría a rellenar siempre esa fecha.
3. CHECK con una lista de valores permitidos (por ejemplo, estado IN ('pendiente','entregado','retrasado')) — no arregla las mayúsculas por sí solo, pero limita qué valores son válidos.
4. CHECK (dias_prestamo > 0): impediría guardar valores negativos o cero.

ALTER TABLE: modificar lo que ya has creado

- En la vida real casi nunca aciertas la tabla a la primera: ALTER TABLE te deja cambiarla sin borrar los datos que ya tiene.
- Puedes añadir una columna nueva, eliminar una que ya no usas, cambiar el tipo de una existente o añadir una restricción a posteriori.
- Ojo: si añades una restricción NOT NULL a una columna que ya tiene filas con NULL, el SGBD te lo va a rechazar hasta que arregles esos datos primero.

ALTER TABLE

```
ALTER TABLE alumno  
  ADD COLUMN telefono VARCHAR(9);
```

```
ALTER TABLE alumno  
  DROP COLUMN fecha_nac;
```

```
ALTER TABLE alumno  
  ADD CONSTRAINT chk_nota  
  CHECK (nota_media BETWEEN 0 AND 10);
```

DROP TABLE y el efecto CASCADE

- DROP TABLE borra la tabla entera, estructura y datos, sin vuelta atrás: no hay papelera de reciclaje.
- Si otra tabla tiene una FOREIGN KEY apuntando a la que quieres borrar, el SGBD te lo impide por defecto, para no dejar relaciones colgando.
- ON DELETE CASCADE, definido al crear la clave ajena, hace que al borrar la fila "padre" se borren automáticamente las filas "hijas" relacionadas — hay que usarlo con mucho cuidado, porque borra en silencio.

DROP TABLE y ON DELETE CASCADE

```
DROP TABLE matricula;
```

```
-- Si al crear la FK definiste CASCADE:
```

```
CREATE TABLE matricula (  
    id_alumno INT,  
    id_modulo VARCHAR(4),  
    PRIMARY KEY (id_alumno, id_modulo),  
    FOREIGN KEY (id_alumno)  
        REFERENCES alumno(id_alumno)  
    ON DELETE CASCADE  
);
```

```
-- Borrar el alumno borra también sus matrículas  
DELETE FROM alumno WHERE id_alumno = 5;
```

¿CASCADE sí o CASCADE no?

Decide si tiene sentido usar ON DELETE CASCADE en cada caso, y por qué.

- 1 Si borras un alumno, ¿debería borrarse en cascada su expediente académico completo?
- 2 Un profesor se da de baja del centro. ¿Tiene sentido que se borren en cascada todas las notas que ha puesto a sus alumnos?
- 3 Intentas hacer DROP TABLE de una tabla ciclo que tiene alumnos apuntando a ella con FOREIGN KEY, y no definiste CASCADE. ¿Qué pasa?

SOLUCIÓN

1. Normalmente no: se suele preferir "desactivar" al alumno (activo = FALSE) antes que borrar en cascada todo su historial académico.
2. No: las notas puestas por ese profesor deben conservarse asociadas a los alumnos, aunque el profesor ya no esté. Aquí no interesa CASCADE.
3. El SGBD rechaza el DROP TABLE con un error de restricción de clave ajena, porque borrarla dejaría filas de alumno apuntando a una tabla que ya no existe.

CREATE INDEX: cuándo ayuda y cuándo estorba

- Un índice es como el índice de un libro: en vez de leer página a página, saltas directo a donde está lo que buscas.
- Merece la pena crearlo sobre columnas que usas mucho en WHERE, JOIN u ORDER BY, sobre todo en tablas grandes.
- Pero tiene un coste: cada índice ocupa espacio y ralentiza un poco los INSERT, UPDATE y DELETE, porque también hay que actualizarlo a él.
- Por eso no se indexa "todo": se indexan las columnas que de verdad se consultan mucho, no las que casi nunca aparecen en un WHERE.

CREATE INDEX

```
CREATE INDEX idx_alumno_ciclo
ON alumno(id_ciclo);

-- Bueno para:
SELECT * FROM alumno WHERE id_ciclo = 3;

-- Quitarlo si ya no aporta:
DROP INDEX idx_alumno_ciclo ON alumno;
```

¿Le pones índice o no?

Para cada situación, decide si merece la pena crear un índice y justifica tu respuesta.

- 1 Una tabla ALUMNO con 50.000 filas, donde el profesorado busca constantemente por apellidos.
- 2 Una tabla CONFIGURACION con solo 5 filas que casi nunca cambian ni se consultan por columnas concretas.
- 3 Una tabla MATRICULA donde se hacen miles de INSERT cada día, pero casi nunca se consulta por la columna fecha.

SOLUCIÓN

1. Sí: tabla grande + columna muy consultada es el candidato perfecto para un índice.
2. No: es tan pequeña que un índice no aporta nada y solo añade complejidad de más.
3. No: si casi nunca se consulta por esa columna, el índice solo ralentizaría los muchos INSERT sin aportar ningún beneficio real.

NULL y la lógica de tres valores

NULL no es un valor como los demás: significa "se desconoce" o "no aplica", y eso cambia las reglas del juego al comparar datos.

- En lógica normal una condición es VERDADERA o FALSA. Con NULL aparece una tercera opción: DESCONOCIDO.
- telefono = NULL nunca es TRUE, aunque la fila tenga el teléfono vacío — por eso nunca se usa = NULL para buscarlos.
- Para preguntar por un NULL se usan operadores especiales: IS NULL e IS NOT NULL.
- NULL también "contagia" las operaciones: 5 + NULL da NULL, y las funciones de agregación como SUM o AVG lo ignoran, no lo cuentan como 0.
- En un WHERE, una fila con resultado DESCONOCIDO se descarta igual que si fuera FALSE — por eso un NOT IN con un NULL de por medio puede dar resultados sorprendentes.

Pon a prueba tu lógica con NULL

Piensa cada respuesta con calma: NULL es la trampa favorita de los exámenes de bases de datos.

- 1 ¿Qué devuelve la condición WHERE telefono = NULL si telefono vale NULL en esa fila?
- 2 ¿Cómo se escribe correctamente la condición para encontrar todos los alumnos sin teléfono registrado?
- 3 Si tienes 10 filas y en la columna nota_media 2 de ellas son NULL, ¿cuántas filas cuenta AVG(nota_media) para calcular la media?
- 4 ¿Por qué SELECT * FROM alumno WHERE id_ciclo NOT IN (1, 2, NULL) puede devolver 0 filas aunque haya alumnos con id_ciclo distinto de 1 y 2?

SOLUCIÓN

1. Nunca es TRUE (es DESCONOCIDO), así que esa fila jamás aparece en el resultado, aunque el teléfono esté vacío.
2. WHERE telefono IS NULL.
3. Solo las 8 filas con valor real: las funciones de agregación ignoran los NULL, no los tratan como 0.
4. Porque comparar con el NULL dentro del NOT IN convierte toda la condición en DESCONOCIDO para cada fila, así que ninguna fila pasa el filtro — una de las trampas más típicas de NULL.

Vistas: tablas que en realidad no lo son

- Una vista parece una tabla, pero por dentro solo es una consulta guardada con nombre.
- Te ahorra reescribir consultas larguísimas una y otra vez, y te permite enseñar solo ciertos datos a según qué usuario.
- La creas una vez con CREATE VIEW y luego la consultas como si fuera una tabla normal.
- Truco: una vista simple (sin agrupar, sin JOIN, basada en una sola tabla) suele ser actualizable — puedes hacer INSERT o UPDATE a través de ella. Una vista compleja, normalmente no.

CREATE VIEW

```
CREATE VIEW alumnos_dam AS
SELECT id_alumno, nombre, apellidos
FROM alumno
WHERE id_ciclo = 1;

SELECT * FROM alumnos_dam;
```

Escribe tu propia vista

La secretaría del centro necesita ver, sin tocar nunca los datos sensibles como el email, solo el nombre, apellidos y ciclo de los alumnos activos (activo = TRUE).

EJERCICIO

Escribe la sentencia CREATE VIEW que resuelve esto, llamando a la vista alumnos_activos.

SOLUCIÓN

```
CREATE VIEW alumnos_activos AS
SELECT nombre, apellidos, id_ciclo
FROM alumno
WHERE activo = TRUE;
```

Usuarios y privilegios

- No todo el mundo tiene por qué poder hacer lo mismo en la base de datos, igual que no todos tus contactos ven el mismo contenido en redes sociales.
- GRANT da permisos (SELECT, INSERT, UPDATE, DELETE...) a un usuario sobre una tabla o vista.
- REVOKE se los quita.
- Regla de oro: dar solo el permiso mínimo necesario. Ni más, ni menos. Esto conecta directamente con la protección de datos del capítulo 1.

Usuarios y permisos

```
CREATE USER 'profesor'@'%'  
  IDENTIFIED BY 'clave_segura';
```

```
GRANT SELECT, INSERT, UPDATE  
ON instituto.alumno  
TO 'profesor'@'%';
```

```
REVOKE DELETE  
ON instituto.alumno  
FROM 'profesor'@'%';
```

¿Qué permisos le das a cada uno?

El instituto tiene distintos perfiles de usuario para la base de datos académica. Decide qué GRANT necesita cada uno, aplicando el mínimo privilegio.

- 1 Un profesor debe poder ver y actualizar las notas de sus alumnos, pero nunca borrar un alumno de la base de datos.
- 2 Un alumno debe poder consultar únicamente sus propias notas, sin tocar nada.
- 3 La secretaría necesita dar de alta nuevos alumnos y corregir datos mal introducidos, pero no debería ver las notas.
- 4 El administrador del sistema necesita poder hacer absolutamente cualquier operación sobre cualquier tabla.

SOLUCIÓN

1. GRANT SELECT, UPDATE ON notas TO profesor — nunca DELETE ni sobre la tabla alumno.
2. GRANT SELECT ON vista_mis_notas TO alumno — mejor aún a través de una vista filtrada por su propio id, no directamente sobre la tabla.
3. GRANT SELECT, INSERT, UPDATE ON alumno TO secretaria — sin conceder ningún permiso sobre la tabla notas.
4. GRANT ALL PRIVILEGES ON *.* TO admin — el único caso donde tiene sentido dar acceso total.

Roles: la plantilla de permisos

- Si tienes 20 profesores, no vas a escribir los mismos GRANT 20 veces: creas un rol una vez y se lo asignas a cada usuario.
- Un rol es como una "plantilla" de permisos: agrupa varios privilegios bajo un nombre y luego lo repartes.
- Si mañana cambia lo que puede hacer un profesor, actualizas el rol una sola vez y afecta a todos los usuarios que lo tienen asignado — mucho menos margen de error que ir tocando usuario a usuario.

CREATE ROLE

```
CREATE ROLE profesor_rol;

GRANT SELECT, INSERT, UPDATE
ON instituto.notas
TO profesor_rol;

GRANT profesor_rol
TO 'profesor1'@'%', 'profesor2'@'%';
```

Roles en el instituto

Decide la solución más rápida y segura en cada situación, pensando en cuántos usuarios hay que mantener.

- 1 El centro contrata a 15 profesores nuevos este curso. ¿Qué es más rápido y seguro: darles GRANT uno a uno o crear un rol?
- 2 Un profesor pasa a ser jefe de departamento y necesita permisos extra. ¿Le añades permisos individuales encima del rol, o creas un rol nuevo "jefe_departamento"?

SOLUCIÓN

1. Crear un rol profesor_rol una vez y asignárselo a los 15 — mismo resultado, muchísimo menos trabajo y menos posibilidad de error.
2. Lo más limpio y escalable es un rol nuevo jefe_departamento que incluya los permisos de profesor_rol más los extra, en vez de ir apilando permisos sueltos difíciles de rastrear con el tiempo.

Las sentencias que más vas a usar

No hace falta memorizarlas todas de golpe: con la práctica se te quedan solas.

Sentencia	Familia	Para qué sirve
CREATE TABLE / VIEW	DDL	Crear tablas, vistas u otros objetos
ALTER TABLE	DDL	Cambiar la estructura de una tabla que ya existe
DROP TABLE / VIEW	DDL	Borrar un objeto de la base de datos (ojo con CASCADE)
CREATE INDEX / DROP INDEX	DDL	Acelerar búsquedas sobre columnas muy consultadas
GRANT / REVOKE	DCL	Dar o quitar permisos a usuarios o roles
CREATE USER / DROP USER	DCL	Crear o eliminar un usuario en el SGBD
CREATE ROLE	DCL	Agrupar permisos en una plantilla reutilizable

Al acabar este bloque tienes que ser capaz de...

- Pasar de "tabla, fila, columna" a "relación, tupla, atributo" sin despeinarte (y al revés), y explicar las reglas que hacen que una tabla sea una relación válida.
- Elegir el tipo de dato correcto para cada columna y crear tablas con CREATE TABLE, incluidas claves primarias simples y compuestas.
- Modificar una tabla ya existente con ALTER TABLE, y borrarla con DROP TABLE entendiendo cuándo actúa ON DELETE CASCADE.
- Decidir cuándo merece la pena crear un índice con CREATE INDEX y cuándo solo añade coste sin beneficio.
- Meter restricciones (NOT NULL, UNIQUE, CHECK, DEFAULT) combinadas en una misma tabla y manejar con soltura la lógica de tres valores de NULL.
- Crear vistas para simplificar y proteger el acceso a los datos, y saber cuándo una vista es actualizable.
- Dar de alta usuarios, agrupar permisos en roles y controlar el acceso con GRANT y REVOKE, aplicando el principio de mínimo privilegio.
- Esto se corresponde con el RA2 y sus criterios de evaluación a) a h) del RD 405/2023.